Integration Scenario Registry

Technical Specification

A read/write registry of Interoperability Implementations for ArchivesSpace, DSpace, CollectionSpace, VIVO, and Fedora

Document Status: Final
Version: 0.4
Date: July 2026
Source Story: F1
Project: LYRASIS Interoperability Project
Systems: Registry Web UI, Registry API, ArchivesSpace, DSpace, CollectionSpace, VIVO, Fedora

Purpose and Scope
Background
Actors
Canonical Data Contract
REST API Contract (required behavior)
Data Model
Email Notifications
Implementation: GitHub-native registry
Behavior Scenarios
Error Scenarios
Other Requirements
Implementation Gaps

Purpose and Scope

This specification defines requirements for an Integration Scenario Registry. The registry is a lightweight repository for storing, discovering, and retrieving records that describe real-world implementations of integrations with ArchivesSpace, DSpace, CollectionSpace, VIVO, and Fedora, or an external system and any of these applications called Community Supported Technologies (CSTs).

The registry serves two primary use cases:

- Deposit (write): A practitioner who has implemented an integration submits a structured record describing what they built, how it works, and how others could replicate it.
- Search and retrieval (read): Any person wanting to find a relevant integration looks up records by system, protocol, integration type, or keyword, and retrieves enough information to evaluate and apply the scenario to their own environment.

The registry is not a code repository, a ticketing system, or a specification authoring tool. It does not host or maintain documentation. It is a discovery and knowledge-sharing layer that links to external resources (code, documentation, specifications) rather than hosting them directly.

Out of scope: hosting integration code or binaries, building or hosting a specific implementation, AI prompt document generation (out of scope for this draft), Export-to-integration-guide crawler (out of scope for this draft; BS01 stretch goal), Link health monitoring cron (noted as future enhancement)

Background

Libraries, archives, and museums using ArchivesSpace, DSpace, CollectionSpace, VIVO, and Fedora sometimes face similar interoperability challenges. Institutions independently solve problems that others have already solved by building custom scripts, using middleware, or implementing plugins. Sometimes this work is documented publicly, but even if so, there is no shared place to go to find out what has already been implemented and how.

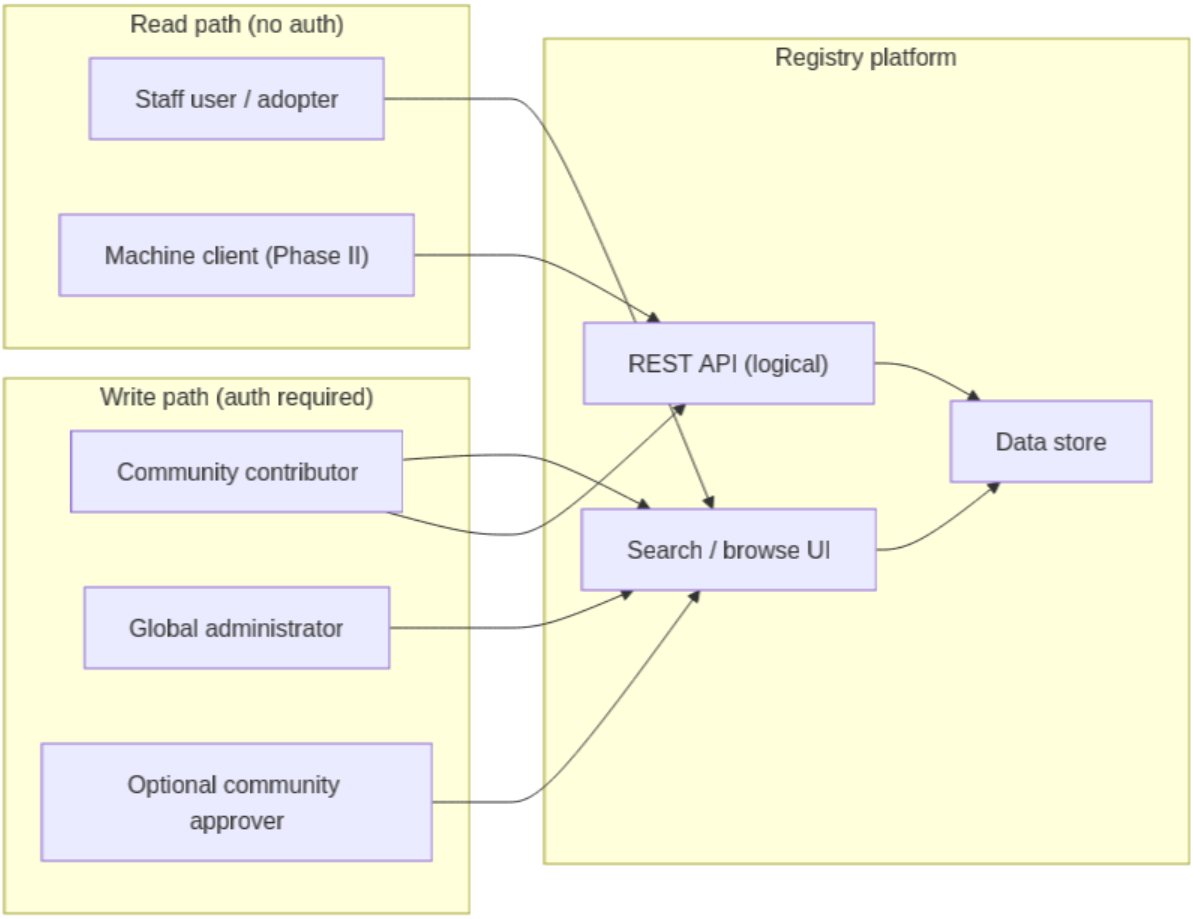
A registry of implemented integration scenarios would reduce duplicated effort, lower the barrier to adopting best-practice integrations, and enable the community to identify patterns and gaps across the ecosystem. It would also complement the technical specifications produced by the LYRASIS Interoperability Project by grounding them in real-world evidence of what has been built.

The registry is designed to be useful to human researchers (browsing via a web interface) in Phase I and automated clients (querying via an API) in Phase II, reflecting the reality that integrations themselves are often machine-to-machine workflows.

Actors

Actor	Role	Notes
Public user	Filters by source/target system; opens a record; follows outbound links (BS01).	No account required for read access.

Actor	Role	Notes
Community contributor	Submits new integration scenario records and updates records they own.	Must have a valid account in the registry. User-Submitted Record Approval. See G-03 for questions about this role. Can make pull requests or submit issues.
Registry Reviewer	Community member who reviews and approves records after they are submitted.	Must be comfortable with YAML. Can merge pull requests.
Global Registry Administrator	Approves accounts, sets user permissions, manages vocabularies, flags records, and monitors system health.	
Machine client	Queries `GET /api/v1/scenarios` with filters (Phase II).	
Registry platform	Persists records, enforces schema, serves UI and API.	



Canonical Data Contract

All platform options below MUST be able to produce and consume this JSON shape.

IntegrationScenarioRecord (JSON)

```json
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "ArchivesSpace digital object linking to DSpace via REST API",
  "source_system": ["ArchivesSpace"],
  "target_system": ["DSpace"],
  "integration_type": "Linking",
  "protocol": ["REST API"],
  "status": "Active",
```

```
  "description": "Bidirectional URI linking between AS digital objects and DSpace
items using REST APIs on both sides.",
  "submitted_by": "Example University Library",
  "submitted_date": "2026-05-15T14:30:00Z",
  "prerequisites": "ArchivesSpace 3.4+, DSpace 7.x with REST enabled, service
accounts on both systems.",
  "configuration_notes": "Configure `dc.identifier.uri` on DSpace; append
`file_versions.file_uri` on AS.",
  "code_repository_url": "https://github.com/example/as-dspace-linker",
  "documentation_url": "https://wiki.example.edu/integrations/as-dspace",
  "related_spec_url":
"https://github.com/lyrasisorghome/InteroperabilityProject/blob/main/specs/A2-dspa
ce-bulk-linking.md",
  "related_scenario_ids": [],
  "min_system_version": ["ArchivesSpace 3.4", "DSpace 7.3"],
  "def_system_version": ["ArchivesSpace 3.5", "DSpace 9.0"],
  "last_verified_date": "2026-04-01",
  "contact_email": "integrations@example.edu",
  "license": "CC-BY-4.0",
  "tags": ["finding aid", "digitization", "bidirectional"],
  "updated_date": "2026-05-20T09:00:00Z",
  "record_url":
"https://registry.example.org/scenarios/550e8400-e29b-41d4-a716-446655440000",
  "flagged": false
}
```


Controlled Vocabularies

Field	Allowed values (initial)
`source_system`, `target_system`	`ArchivesSpace`, `DSpace`, `CollectionSpace`, `VIVO`, `Fedora`, `Other` (+ extensible)
`integration_type`	`Linking`, `Deposit`, `Metadata Harvesting (OAI-PMH)`, `Search/Discovery`, `Bidirectional Sync`, `Other`
`protocol`	`SWORD v2`, `SWORD v3`, `OAI-PMH v2`, `REST API`, `SPARQL`, `Custom/Bespoke`, `Other`

Field	Allowed values (initial)
`status`	`Active`, `Experimental`, `Deprecated`, `Retired`, `Recalled`

Multi-select fields are JSON arrays. Admin-maintained vocabularies are exposed at `GET /api/v1/vocabularies`.

Duplicate detection (edge case)

On `POST /api/v1/scenarios`, the registry SHOULD evaluate:

Rule	Match	Action
Hard advisory	Same `submitted_by` + same sorted `source_system` + same sorted `target_system` + same `integration_type`	Return `409` with `duplicate_of` UUID **or** `201` with `warnings[]` (configurable by admin)
Soft advisory	Levenshtein similarity on `title` ≥ 0.85 vs existing Active/Experimental records	Return `201` with `warnings[]`; do not block

Default for Phase I: **warn, do not block** (this is a community registry, not cataloging authority).

REST API Contract (required behavior)

Implementations MAY defer HTTP exposure to Phase II, but Phase I storage and UI MUST be mappable to these operations.

Method	Endpoint	Auth	Description
GET	`/api/v1/scenarios`	No	List/search records (query params below)
POST	`/api/v1/scenarios`	Yes	Create record; server assigns `id`,

Method	Endpoint	Auth	Description
			`submitted_date`, `record_url`
GET	`/api/v1/scenarios/{id}`	No	Single record
PATCH	`/api/v1/scenarios/{id}`	Yes (owner or admin)	JSON Merge Patch partial update
GET	`/api/v1/vocabularies`	No	Controlled lists
GET	`/api/v1/schema`	No	JSON Schema for records

Search query parameters (`GET /api/v1/scenarios`):

Param	Example	Semantics
`source_system`	`ArchivesSpace`	Record's `source_system` contains value
`target_system`	`DSpace`	Record's `target_system` contains value
`integration_type`	`Linking`	Exact match
`protocol`	`REST API`	Record's `protocol` contains value
`status`	`Active`	Exact match; default public UI excludes `Retired`, `Recalled`
`keyword`	`finding aid`	Full-text on `title`, `description`, `tags`
`tags`	`digitization`	Any tag match
`updated_since`	`2026-01-01T00:00:00Z`	`updated_date` ≥ value

Param	Example	Semantics
`page`, `per_page`	`1`, `20`	Pagination; default `per_page=20`, max `100`

Example: search response

None

```
GET
/api/v1/scenarios?source_system=Fedora&target_system=DSpace&status=Active&page=1&per_page=2
```

JSON

```
{
  "data": [
    {
      "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "title": "Fedora object metadata sync to DSpace via OAI-PMH",
      "source_system": ["Fedora"],
      "target_system": ["DSpace"],
      "integration_type": "Metadata Harvesting (OAI-PMH)",
      "protocol": ["OAI-PMH v2"],
      "status": "Active",
      "description": "…",
      "record_url":
"https://registry.example.org/scenarios/7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "updated_date": "2026-03-10T11:00:00Z"
    }
  ],
  "meta": {
    "page": 1,
    "per_page": 2,
    "total": 14,
    "total_pages": 7
  },
  "links": {
    "self": "/api/v1/scenarios?source_system=Fedora&target_system=DSpace&page=1",
    "next": "/api/v1/scenarios?source_system=Fedora&target_system=DSpace&page=2"
  }
}
```

Example: create request

```
None
POST /api/v1/scenarios
Authorization: Bearer {token}
Content-Type: application/json
```

```
JSON
{
  "title": "ArchivesSpace to Archive-It crawl configuration",
  "source_system": ["ArchivesSpace"],
  "target_system": ["Other"],
  "integration_type": "Linking",
  "protocol": ["Custom/Bespoke"],
  "status": "Experimental",
  "description": "Export finding aid URLs from AS for Archive-It seed list.",
  "submitted_by": "State Archives",
  "documentation_url": "https://example.org/docs/as-archive-it",
  "tags": ["web archiving", "Archive-It"]
}
```

Example success response (`201 Created`):

```
JSON
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "title": "ArchivesSpace to Archive-It crawl configuration",
  "source_system": ["ArchivesSpace"],
  "target_system": ["Other"],
  "integration_type": "Linking",
  "protocol": ["Custom/Bespoke"],
  "status": "Experimental",
  "description": "Export finding aid URLs from AS for Archive-It seed list.",
  "submitted_by": "State Archives",
  "submitted_date": "2026-06-06T16:00:00Z",
  "documentation_url": "https://example.org/docs/as-archive-it",
  "tags": ["web archiving", "Archive-It"],
  "updated_date": "2026-06-06T16:00:00Z",
```

```
  "record_url":
"https://registry.example.org/scenarios/a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "flagged": false,
  "warnings": []
}
```

Example: validation error (422)

```
None
{
  "error": "validation_failed",
  "message": "One or more required fields are missing or invalid.",
  "details": [
    { "field": "source_system", "code": "required", "message": "Must include at
least one system." },
    { "field": "integration_type", "code": "invalid_enum", "message": "Must be one
of: Linking, Deposit, …" }
  ]
}
```

Example: unauthorized edit (403)

```
None
{
  "error": "forbidden",
  "message": "Authenticated user is not the record owner or an administrator."
}
```

Rate limiting: Public read: 120 requests/minute per IP; authenticated write: 30/minute. Respond `429` with `Retry-After` header.

Data Model

Required Fields

Field	Type	Description
id	UUID	System-generated unique identifier. Stable and permanent once assigned.
title	Text	Short, descriptive name for the integration (e.g., 'ArchivesSpace Digital Object linking to DSpace via REST API').
source_system	Controlled list (multi-select)	One or more of: ArchivesSpace, DSpace, CollectionSpace, VIVO, Fedora, Other. The system(s) where the integration originates or is initiated.
target_system	Controlled list (multi-select)	One or more of the same list. The system(s) that receive data or expose endpoints consumed by the integration.
integration_type	Controlled list	One of: Linking, Deposit, Metadata Harvesting (OAI-PMH), Search/Discovery, Bidirectional Sync, Other.
protocol	Controlled list (multi-select)	One or more of: SWORD v2, SWORD v3, OAI-PMH v2, REST API, SPARQL, Custom/Bespoke, Other.
status	Controlled list	One of: Active (currently in use at the submitting institution), Experimental (proof of concept), Deprecated (was active, no longer maintained), Retired, Recalled.
description	Long text	Free-text description of what the integration does, the problem it solves, and any key implementation decisions. Markdown supported.
submitted_by	Text / User ref	Institution name (and optionally a contact name) of the submitter.
submitted_date	Date	System-generated submission timestamp (ISO 8601).

Optional Fields

Field	Type	Description
prerequisites	Long text	Software versions, permissions, configurations, or third-party tools required before the integration can be replicated.

Field	Type	Description
configuration_notes	Long text	Practical notes for replication: key configuration parameters, known gotchas, institution-specific decisions. Markdown supported.
code_repository_url	URL	Link to a public code repository (GitHub, GitLab, etc.) containing implementation code, scripts, or plugins.
documentation_url	URL	Link to external documentation, wiki page, or technical write-up.
related_spec_url	URL	Link to a formal technical specification (e.g., a LYRASIS Interoperability Project spec) that this implementation corresponds to.
related_scenario_ids	UUID list	IDs of other registry records that this scenario builds on, extends, or is related to.
source_system_profile	URL	Link to IOI InfraFinder, COAR IRD, or other system profile. Implementation recommendation: Show links to these registries in the UI.
target_system_profile	URL	Link to IOI InfraFinder, COAR IRD, or other system profile. Implementation recommendation: Show links to these registries in the UI.
min_system_versions	Free text (array)	Minimum version of source/target systems tested (e.g., 'ArchivesSpace 3.4, DSpace 7.3').
def_system_version	Controlled List (multi-select)	Main version of source/target systems tested
last_verified_date	Date	Date the implementer last confirmed the integration was still operational.
github_id	URL	Optional contact for follow-up questions.
license	SPDX identifier	License under which the configuration notes and linked code are shared (e.g., CC-BY-4.0, MIT, Apache-2.0).
tags	Text list (free)	Free-text keywords for discovery (e.g., 'digitization workflow', 'born digital', 'finding aid', 'researcher profile').
updated_date	Date	System-generated timestamp of the most recent edit to the record.
edited_by	Free text	User input name or username

System-Controlled Fields (not editable by depositor)

Field	Type	Description
record_url	URL	Canonical URL for this record in the registry (constructed from base URL + id).
version_history	Audit log	Immutable log of all edits, with timestamp and editor, retained for provenance. [PLACEHOLDER — versioning model TBD, see Gap G-05.]
flagged	Boolean	Set by registry administrator if a record is under review for accuracy or policy issues.

Deletions

Records that are marked inactive in the registry are removed from default public views of the registry. No records are deleted from the registry.

Email Notifications

The registry will send email directly. Several events will trigger notifications in GitHub, including:

- Account access request
- Record submitted
- Record updated
- Record status change

Anyone who has access to the repository will receive notifications based on their own configurations. Users can follow pull requests to receive email updates.

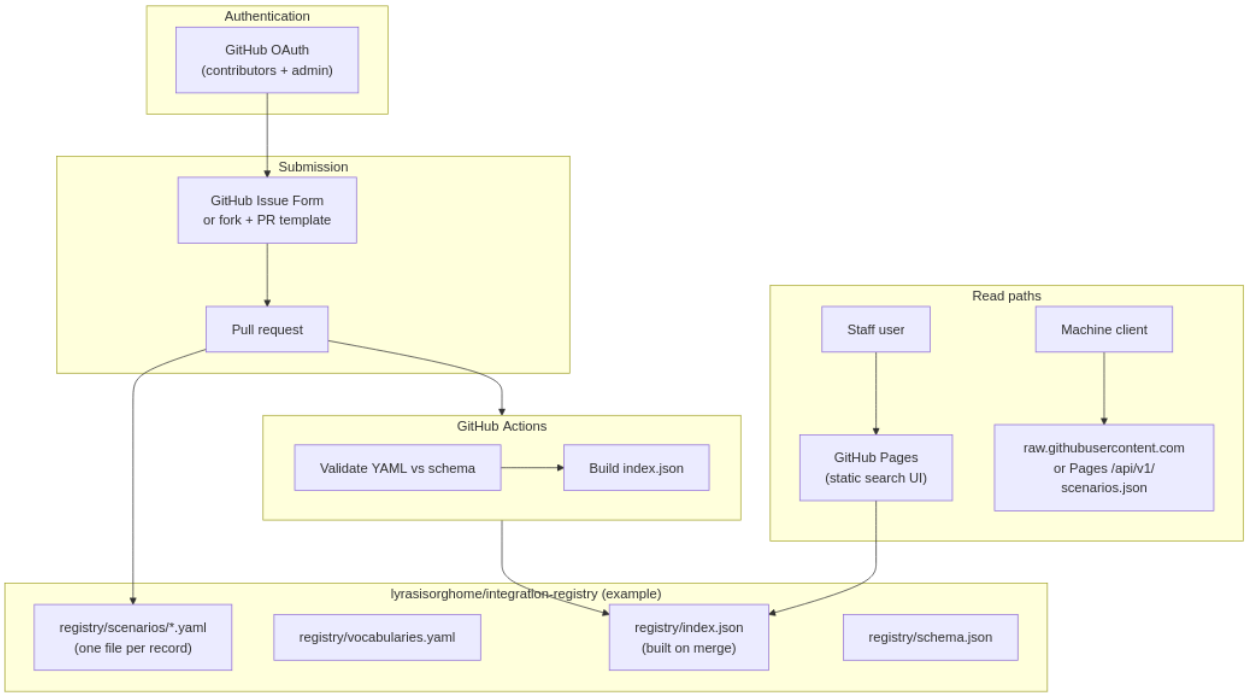
Implementation: GitHub-native registry

This implementation option was selected by the product spec team, which included representatives from all five Lyrasis CSTs. The consultants also drafted two alternative implementations (see here).

Best for: GitHub SSO alignment, PR-based review, version control, LYRASIS already on GitHub, zero custom server for Phase I.

Weak fit for: Non-technical contributors who will not use GitHub; WYSIWYG markdown editing expectations.

Architecture



Submission via pull request

This is a common pattern, human YAML contribution:

1. Contributor forks repo (or uses GitHub web editor with branch).
2. Adds `registry/scenarios/{uuid}.yaml` using PR template checklist.
3. GitHub Action validates against `registry/schema.json` (JSON Schema).
4. Admin (or optional community approver) reviews PR; merge = publish.
5. Action rebuilds `registry/index.json` for search UI and API consumers.

GitHub Issue Form contribution opens an issue with structured fields; a bot or maintainer converts approved issues to YAML via PR (lower barrier for first-time contributors).

Example record file

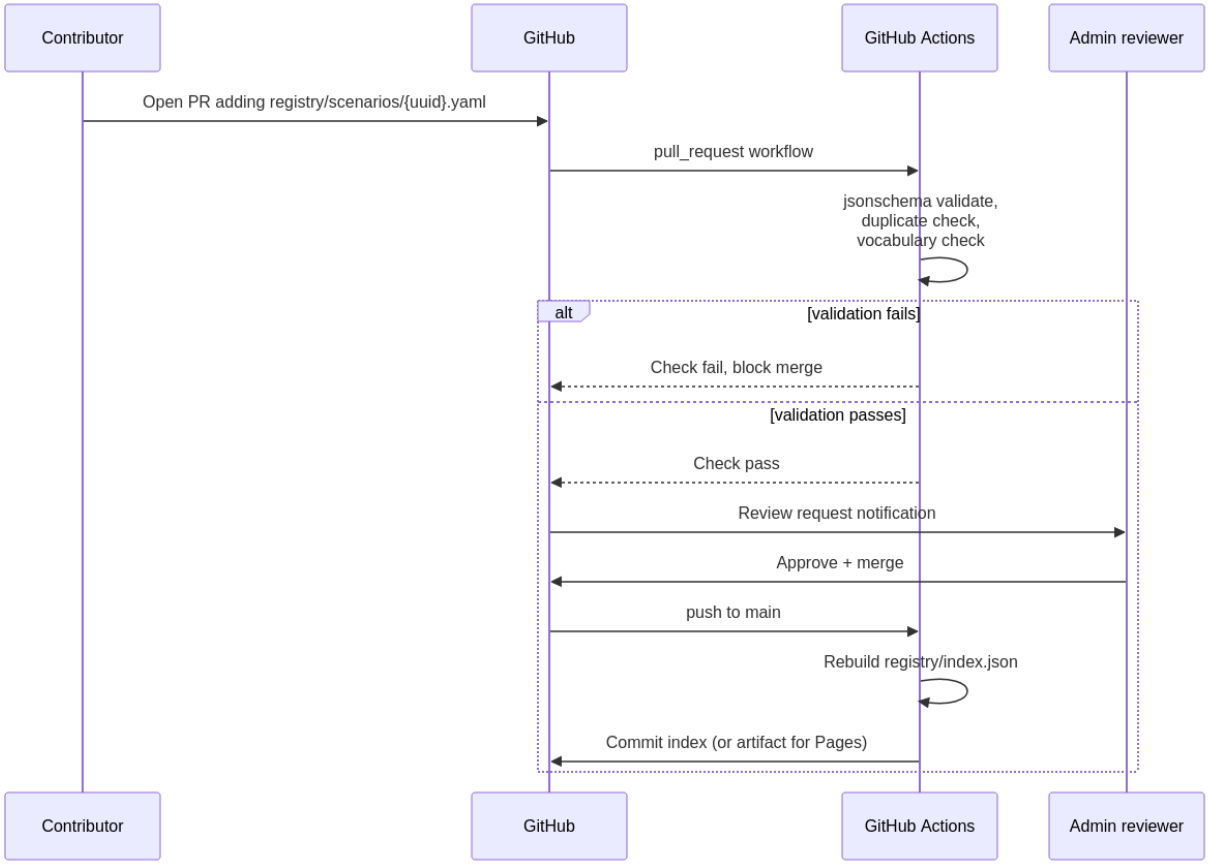
`registry/scenarios/550e8400-e29b-41d4-a716-446655440000.yaml`:

```
None
id: 550e8400-e29b-41d4-a716-446655440000
title: ArchivesSpace digital object linking to DSpace via REST API
source_system:
```

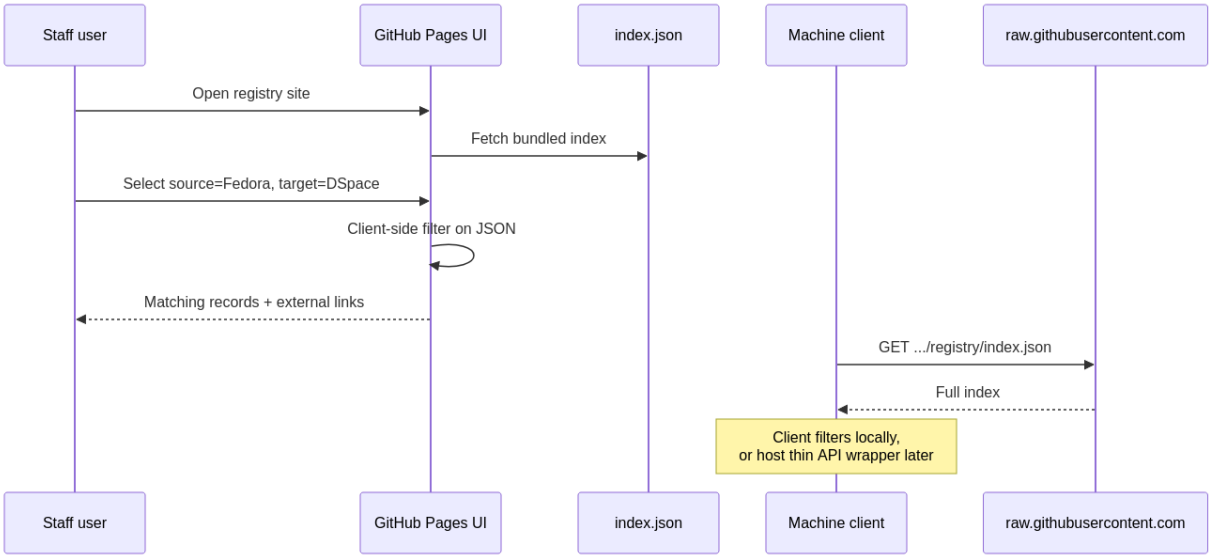
```
  - ArchivesSpace
target_system:
  - DSpace
integration_type: Linking
protocol:
  - REST API
status: Active
description: |
  Bidirectional URI linking between AS digital objects and DSpace items.
submitted_by: Example University Library
submitted_date: "2026-05-15T14:30:00Z"
code_repository_url: https://github.com/example/as-dspace-linker
related_spec_url:
https://github.com/lyrasisorghome/InteroperabilityProject/blob/main/specs/A2-dspac
e-bulk-linking.md
tags:
  - finding aid
  - digitization
updated_date: "2026-05-20T09:00:00Z"
flagged: false
```

`git log` on each file = **version history** (resolves parent G-05). `status: Retired` removes from default index build (no delete).

Deposit sequence



Search sequence



Phase II upgrade path: add Cloudflare Worker or Lambda that serves `GET /api/v1/scenarios` with server-side filter over the same JSON index — **no schema change**.

Moderation and Review

Users may submit records to the registry after logging in through GitHub.

The registry should include [Community] Approver or Reviewer roles that communities can choose to implement. Anyone with an authenticated account could be assigned these roles. Users assigned with this role get email notifications from the registry when new records are submitted, with a link to review the record. The Approver can then update status to Recalled if the record needs to be reviewed for potential spam or other harmful material.

Static UI sketch

Minimal Phase I UI (Jekyll/Eleventy/React Single-Page Application (SPA) on Pages):

- Dropdowns: source system, target system, integration type, protocol, status
- Free-text keyword box
- Results table: title, systems, status, links
- Record detail page: render YAML fields + markdown `description`

Embed link from Fedora Confluence / project site (per BS01 navigation — exact URL is a content decision, not technical).

Tradeoffs

Pros	Cons
Native GitHub SSO + PR review	Contributors must tolerate GitHub (or issue form + maintainer proxy)
Full version history, public audit	No built-in email; use GitHub notifications
Same org as InteroperabilityProject	Custom search UI is a small dev task
Seed records can reference specs in sibling repo	Immediate publish on merge (optional approver = merge gate)
Phase II API = index.json + thin wrapper	

How requirements map: GitHub Implementation

Requirement	GitHub implementation
Read	Public GitHub repository; non-authenticated users can view and download records
Spam moderation	The human reviewer in the deposit loop will deny obvious spam content, but there is no automated method in this implementation of keeping people from making PRs with bad data. There are many tools for moderating repository interactions in GitHub repo settings.
Deposit	Two options, both enabled: ● User submits integration scenario via GitHub Issues Form or API > maintainer (or bot) drafts YAML > maintainer (or bot) opens PR > Reviewer (human) validates and merges PR
Validation	A GitHub Action to validate pull requests
Search UI	index.json is available for developers to create a simple single-page search apps GitHub has some basic search functionality, but can't support use cases such as filtering to "Fedora > DSpace integrations"

Requirement	GitHub implementation
Controlled vocabularies	Referenced by Forms to fill dropdowns. Form will be created in implementation and must be maintained/updated by an administrator, including controlled vocabularies.
Admin approval of users	Repository > Settings > Access: Collaborators and Teams allows admins to set and define roles; and to add, remove, and accept users.
GitHub SSO	Native GitHub login workflow
Version history	Via UI: GitHub log; Pull Request history for review context `git log registry/scenarios/{uuid}.yaml`
Email notifications	Repository > Settings > Email Notifications allows the admin to set up email addresses to receive notifications when push events are triggered. GitHub users can define their own much more granular notifications at Profile > Notifications button > Manage notifications
Public read API	Phase I does not include a public read API. Phase II could use GitHub REST API
UUID `id`	Server assigns UUID with GitHub Actions on pull requests. Reviewer validates before merging to enforce file name matching.
No delete	Records with status `Retired` or `Recalled` are excluded from `index.json` but remain in the repo for audit history.

POC Sample

https://github.com/ryanttb/LyrasisInteroperabilityProject/tree/f1-poc/registry

Seed content plan (implementation phase)

Populate **10–15 records** as specified in issue #58:

#	Source	Target	Type	Seed from
1	ArchivesSpace	DSpace	Linking	A2-dspace-bulk-linking.md

#	Source	Target	Type	Seed from
2	VIVO	DSpace	Deposit	V1-vivo-sword-deposit.md
3	CollectionSpace	—	OAI-PMH	C1-cs-oai-pmh.md
4	ArchivesSpace	Archive-It	Linking	PoC external integration
5–15	Mix of five CSTs	Cross-CST pairs	Various	CST Interoperability Project data, community submissions, or SME interviews

Behavior Scenarios

Search and Read

BS-01: User can search the registry by filtering records

Step	Description
Given	The user has discovered and accessed the registry URL.
	No login or account is required.
When	The user opens the registry web UI and selects an option to search the registry
Then	The user receives a keyword search bar with result filtering options: source_system, target_system, integration_Type, protocol, status
When	The user selects “Fedora” in the Source System filter and “DSpace” in the Target System filter.
Then	The results list shows all records where source_system includes ArchivesSpace **and** target_system includes DSpace
	Each result card shows: Title, integration type, protocol(s), status, submitting institution, and submission date

Step	Description
	Results are sortable by: date (newest first default), status (Active first), and relevance (if keyword search is also active)
	Result sorting is explained in the UI
	Result filtering options are the same: source_system, target_system, integration_Type, protocol, status

BS-02: User can search the registry by keyword

Step	Description
Given	The user has discovered and accessed the registry URL.
	No login or account is required.
When	The user opens the registry web UI and selects an option to search the registry
Then	The user receives a keyword search bar with result filtering options: source_system, target_system, integration_Type, protocol, status
When	The user types a keyword in the search bar.
Then	The results list shows all records where source_system includes ArchivesSpace **and** target_system includes DSpace
	Each result card shows: Title, integration type, protocol(s), status, submitting institution, and submission date
	Results are sortable by: date (newest first default), status (Active first), and relevance (if keyword search is also active)
	Result sorting is explained in the UI
	Result filtering options are the same: source_system, target_system, integration_Type, protocol, status

BS-03: User can search the registry by UUID

Step	Description
Given	The user has discovered and accessed the registry URL.
	No login or account is required.
When	The user opens the registry web UI and selects an option to search the registry

Step	Description
Then	The user receives a keyword search bar with result filtering options: source_system, target_system, integration_Type, protocol, status
When	The user types the UUID in the keyword search bar
Then	The target one (1) result is returned.

BS-04: User views a full integration scenario record

Step	Description
Given	A search result is displayed in the registry.
When	The researcher clicks the record title.
Then	A full record detail page is displayed, showing all populated fields from Data Model.
	Links to code_repository_url, documentation_url, related_spec_url, and related_scenario_ids are rendered as clickable hyperlinks.
	A 'Copy record URL' button provides the canonical URL for sharing or citing the record.
	A machine-readable link (e.g., a JSON badge or 'View as JSON' link) is displayed for API consumers.

Deposit (Write)

BS-05: User submits a new integration record via web form

Step	Description
Given	The user has an account with the registry platform and is logged in.
	The user has successfully deployed and tested an integration between two or more of the target systems
When	The user opens the registry web UI and selects an option to Submit a New Integration Scenario
	The user completes the required fields and applicable optional fields
	The user previews the record and submits it.
Then	The registry assigns a stable UUID to the record and sets submitted_date to the current timestamp

Step	Description
	The record is published to the registry, available for public access
	An email notification is sent to community reviewer(s) with the new record's canonical URL and issue or pull request.

BS-06: User submits a new integration record via the API

Step	Description
Given	The implementer holds a valid API token (obtained after account creation).
When	The implementer sends an authenticated HTTP POST request to the registry API endpoint with a JSON document conforming to the integration scenario record schema.
Then	The registry validates the submitted JSON against the schema.
	If validation passes: the record is created, a UUID is assigned, and the API returns HTTP 201 with the new record's canonical URL in the Location header and the full record JSON in the response body.
	If validation fails: the API returns HTTP 422 with a structured error response listing the failing fields and reasons (see Error Scenarios).
	The record is published to the registry, available for public access

BS-07: Community reviewer receives an email notification about the new record

Step	Description
Given	The community reviewer has an account with the registry platform at the permission level Community Reviewer.
	The community reviewer receives an email notification with the new record's canonical URL and summary fields
When	The community reviewer follows the record URL
Then	The community reviewer views the full record and has the option to edit the record

Update (Write)

BS-D08: User updates a record they own

Step	Description
Given	The user is logged in and is the owner of an existing record (submitted_by matches their account).
When	The user edits the record via the web UI or sends an authenticated HTTP PATCH request to the record's API endpoint.
	The implementer updates one or more fields (e.g., status changed from Active to Deprecated, last_verified_date updated, system_versions corrected).
Then	The registry saves the changes and updates updated_date to the current timestamp.
	An entry is appended to the record's version_history audit log.
	The updated record is immediately reflected in search results and API responses.

BS-09: Community reviewer updates a record they've reviewed

Step	Description
Given	The community reviewer viewed the full record and has the option to edit the record
When	The community reviewer opens the Edit view, updates the Description field and the Edited_By field (or any other field), and clicks Save
Then	The registry saves the changes and updates updated_date to the current timestamp.
	An entry is appended to the record's version_history audit log.
	The updated record is immediately reflected in search results and API responses.

Export (Harvest)

BS-10: Machine client retrieves all records (full harvest)

Step	Description
Given	A machine client wants to mirror or index the full registry.
When	The client sends GET /api/v1/scenarios with no filters and iterates through paginated responses.
Then	All records are returned across paginated responses. Each page includes a next_page URL in the response body.
	The client can optionally filter by updated_since={ISO 8601 date} to retrieve only records modified after a given timestamp, enabling incremental synchronization.

Error Scenarios

Scenario	HTTP / UX behavior
Deposit missing required fields	`422` + `details[]`; Form/PR template shows inline errors
Duplicate detection	`201` + `warnings[]` or configurable `409`
Search no results	`200` + `"data": []`, UI message: "No integrations match; try broader filters."
Unauthorized edit	`403` + error JSON
Record flagged	Public `GET` returns record with `"flagged": true` banner; or exclude from index (admin config)

Other Requirements

Documentation

The registry requires detailed, public user documentation to ensure adoption that includes:

- Gaining authentication for submitting an integration
- Submitting an integration record
- Updating a record

- Setting up email notifications
- Searching the registry
- PR/Community review process
- Exporting data from the registry
- Troubleshooting
- Governance, support, and contact information
- Accessibility features and compliance
- Reference documentation, such as field definitions

The documentation may include screenshots and examples.

Governance

The Lyrasis Organizational Home will serve as the administrative home for the registry. Gap G-02 notes that during the implementation phase, a decision about who is ultimately responsible for the registry's continued existence should be made.

Product ownership will be shared by the community product teams and community volunteers. Establishing co-governance tools that involve community managers and registry approvers are required for collaboration. Sample tools include:

- a periodic meeting
- A slack channel
- GitHub comments

Changes to the registry should be proposed and discussed in whatever co-governance spaces the community and product implementation teams devise.

Community managers will manage GitHub repository access roles and onboard community users to the tool.

It is recommended that each community appoint a person or group to be the registry entry approvers. See Moderation and Review for more information about how the registry incorporates the review process. How the community appoints their approver may vary depending on the community's structure. Some communities may decide this is incorporated into a technical development or documentation team's charge; others may assign a loose group of interested volunteers, a single individual, or the community's manager to the role.

Security and Data Privacy

Contributors are required to log into GitHub before submitting a record. There are many tools for moderating repository interactions in GitHub repo settings. GitHub's full security features can be found here: https://github.com/security

Records with status `Retired` or `Recalled` are excluded from `index.json` but remain in the repo for audit history.

Performance and Scalability

The spec is designed to create a minimum viable product with the simplest architecture possible. While GitHub has had more downtime than is typical the past year due to increased traffic, it is a large, stable, well-funded platform that is likely to exist into the foreseeable future. The slight performance issue with increased downtime does not warrant self hosting a solution.

For more information on GitHub's ongoing optimization projects, see: https://github.blog/engineering/architecture-optimization/

Implementation Gaps

#	Gap	Description	Owner
G-01	Maintenance plan	Define how records will be deprecated for integrations that are no longer supported or possible (technical solution is to set as Inactive; community process should be defined with program managers).	PM, Communities
G-02	Maintenance plan	Define the administrative home of the new product. Who is ultimately responsible for its continued existence?	Lyrasis leadership/Bridget, PMs
G-03	JSON Schema	A JSON schema is needed as part of the implementation	Implementation team